interview-transcribe — a local AI interview app

WhisperX · Obsidian · gum TUI on Pop!_OS 24.04 LTS with an NVIDIA GPU. Build a private, offline, speaker-aware interview transcription app you launch from your terminal or the app grid.

What you're building

By the end of this guide, you can open a terminal (or click the app launcher), type `interview-transcribe`, and walk through four prompts: pick an audio file, type the stakeholder's name, choose a model size, confirm. A spinner shows progress. When it's done, your transcript — with speaker labels — is sitting in your Obsidian vault inside a templated note, ready to read and synthesize.

The pieces:

- **WhisperX** — Whisper + pyannote.audio for transcription with speaker diarization, GPU-accelerated.
- **A bash engine** (`transcribe-interview`) — runs WhisperX, names the output, drops it into the vault from a template.
- **A gum TUI** (`interview-transcribe`) — interactive front-end that calls the engine.
- **A desktop launcher** — `.desktop` file so the TUI shows up in the COSMIC app grid.
- **Obsidian** (Flatpak from Flathub) — markdown vault for the actual synthesis work.
- **Ollama** (optional) — local LLM for one-paragraph summaries and action-item extraction.

Before you start

Open a terminal (in COSMIC: Super key, type "Terminal") and run the two checks below. If either fails, fix that before continuing.

```
# Confirm Pop!_OS version
lsb_release -a

# Confirm the NVIDIA driver is loaded
nvidia-smi
```

`lsb_release` should report **Pop!_OS 24.04 LTS**. `nvidia-smi` should print a table showing your GPU and driver version. If `nvidia-smi` says "command not found," you booted from the wrong Pop!_OS ISO — reinstall with the **NVIDIA** ISO from system76.com, or run `sudo apt install system76-driver-nvidia` and reboot.

> **Note.** Pop!*OS 24.04 ships with the NVIDIA driver pre-installed if you used the NVIDIA ISO. You do _not* need to install CUDA system-wide — PyTorch bundles its own CUDA runtime via pip. Skip every "install CUDA toolkit" guide you'll find online.

Part 1 — Setup

1. Install system dependencies

WhisperX needs `ffmpeg`, a Python virtual environment toolchain, and **gum** (Charm's shell-script UI toolkit, which we'll use to build the TUI). On Ubuntu 24.04 / Pop!_OS 24.04, Python is "externally managed" (PEP 668) — pip refuses to install into the system Python, so a virtual environment is required, not optional.

```
# Core packages
sudo apt update
sudo apt install -y ffmpeg python3-venv python3-pip git build-essential curl

# Add the Charm apt repo (for gum)
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://repo.charm.sh/apt/gpg.key \
```

```
    | sudo gpg --dearmor -o /etc/apt/keyrings/charm.gpg
echo "deb [signed-by=/etc/apt/keyrings/charm.gpg] https://repo.charm.sh/apt/ * *" \
    | sudo tee /etc/apt/sources.list.d/charm.list
sudo apt update && sudo apt install -y gum

# Verify
ffmpeg -version | head -n 1
python3 --version
gum --version
```

You want `ffmpeg` 6.x or newer, Python 3.12.x, and any modern `gum`.

2. Create a virtual environment for WhisperX

Keep the AI dependencies in one place at `~/.venvs/whisperx`. The path is referenced by the engine script later — change it there if you use a different location.

```
mkdir -p ~/.venvs
python3 -m venv ~/.venvs/whisperx
source ~/.venvs/whisperx/bin/activate

pip install --upgrade pip setuptools wheel
```

Your prompt should now have `(whisperx)` at the front. Every command in the rest of Part 1 assumes that prefix — if you open a new terminal, re-run `source ~/.venvs/whisperx/bin/activate` first.

3. Install PyTorch with CUDA support

Install PyTorch *before* WhisperX so you control the CUDA version. The pip wheels for CUDA 12.x bundle the runtime — you do not need a system CUDA install.

```
pip install torch torchaudio \
    --index-url https://download.pytorch.org/whl/cu124

# Verify CUDA is visible from PyTorch
python <<'PY'
import torch
print('cuda available:', torch.cuda.is_available())
if torch.cuda.is_available():
    print('device:', torch.cuda.get_device_name(0))
PY
```

You should see `cuda available: True` and your GPU name. If it prints `False`, your NVIDIA driver is too old for CUDA 12.4 — run `sudo apt upgrade nvidia-driver-560` (or newer) and reboot.

> **Note.** The PyTorch CUDA wheel index lags upstream CUDA by a few months. cu124 (CUDA 12.4) is the stable target as of May 2026. If pytorch.org shows a newer cu12x line on the install page, use that instead. Avoid `pip install torch` from plain PyPI — it gives you a CPU build.

4. Install WhisperX

```
pip install whisperx

# Confirm the binary is on the venv's PATH
which whisperx
whisperx --help | head -n 20
```

`which whisperx` should print a path inside `~/.venvs/whisperx/bin`. If you see "not found," you forgot to activate the venv (step 2).

5. Hugging Face token and gated-model terms

Speaker diarization uses pyannote.audio models, which are gated — you have to accept usage terms on the Hugging Face website with the same account you'll generate a token for. This is the single most common reason WhisperX setups fail. Do all four steps:

- Create a free account at **huggingface.co** if you don't have one.
- Visit **huggingface.co/pyannote/segmentation-3.0** and click "Agree and access repository."
- Visit **huggingface.co/pyannote/speaker-diarization-3.1** and accept there too.
- Go to **huggingface.co/settings/tokens**, create a new token with the **read** role, and copy it.

Store the token in your shell environment so scripts can read it:

```
# Append to ~/.bashrc (or ~/.zshrc if you've switched shells)
echo 'export HF_TOKEN="hf_your_actual_token_here"' >> ~/.bashrc

source ~/.bashrc
echo $HF_TOKEN     # should print your token
```

Note. If you skip the two "accept terms" pages, the diarization step fails with an error like "Could not download pyannote/speaker-diarization-3.1. Make sure to accept the user conditions on the model page." The token alone is not enough; the same account must have agreed to the terms.

6. First transcription test (raw CLI)

Before wiring up the engine and TUI, confirm the pipeline works end-to-end. Grab any short audio file you have (voice memo, meeting recording).

```
mkdir -p ~/transcripts && cd ~/transcripts
cp /path/to/your/sample.m4a ./sample.m4a

whisperx sample.m4a \
    --model large-v3 \
    --diarize \
    --hf_token $HF_TOKEN \
    --output_format all \
    --compute_type float16
```

First run downloads about 3 GB of model weights — normal. Subsequent runs reuse them from `~/.cache`. When it finishes, you'll have `.srt`, `.vtt`, `.json`, `.tsv`, and `.txt` files next to `sample.m4a`. The TXT file with SPEAKER_NN: prefixes is what the engine uses.

Note. If you have less than 8 GB VRAM, use `--model large-v2 --batch_size 4`. For 4 GB cards, drop to `--model medium --compute_type int8`. Add `--language en` if your interviews are always in English — it skips language detection and starts faster.

7. Install Obsidian

The Flatpak from Flathub is the supported Linux build — Obsidian's own download page links to it, and the maintainer is the Obsidian team. Avoid the random .deb files floating around forums.

```
# Flathub should already be configured on Pop!_OS 24.04. If not:
flatpak remote-add --if-not-exists flathub \
    https://flathub.org/repo/flathub.flatpakrepo

flatpak install -y flathub md.obsidian.Obsidian

# Launch
flatpak run md.obsidian.Obsidian
```

On first launch Obsidian asks you to create or open a vault. Point it at `~/Documents/Vault` (we'll create it in step 8). Once it opens, click the gear icon in the bottom-left to open settings, find the **About** tab, and turn off **Automatic updates** — Flatpak handles updates for you, and Obsidian's built-in updater fights the sandbox.

8. Create the vault structure

A flat vault gets messy fast once you have a dozen interviews. This structure keeps raw transcripts separate from synthesis notes.

```
mkdir -p ~/Documents/Vault/{Interviews,Transcripts,Synthesis,Templates,_attachments}
```

```
# A transcript template
cat > ~/Documents/Vault/Templates/Interview.md <<'EOF'
---
type: interview
stakeholder:
project:
date:
duration:
status: raw
tags: [interview, raw]
---

# Interview — {{stakeholder}} ({{date}})

## Summary


## Key claims


## Open questions


## Action items


---

## Transcript

<!-- Speaker-labelled transcript inserted here by the engine -->
EOF

echo "Vault scaffold created."
ls -la ~/Documents/Vault
```

In Obsidian settings, find the **Core plugins** section and enable the **Templates** plugin. A new **Templates** entry appears further down the sidebar — open it and set the template folder location to `Templates`. Under **Hotkeys**, bind *Insert template* to Ctrl+T.

Part 2 — Build the app

9. The transcription engine

The engine is a bash script that runs WhisperX, names the output file, and drops a templated markdown note into the vault. It's deliberately boring — no UI, no prompts. The TUI calls it and shows progress.

Save the following as `~/.local/bin/transcribe-interview` and `chmod +x` it.

```
mkdir -p ~/.local/bin
cat > ~/.local/bin/transcribe-interview <<'EOF'
#!/usr/bin/env bash
# Engine — called by interview-transcribe TUI or directly.
# Usage:  transcribe-interview <audio-file> <stakeholder> [model]
set -euo pipefail

if [[ $# -lt 2 ]]; then
    echo "usage: transcribe-interview <audio-file> <stakeholder> [model]" >&2
    exit 1
fi

AUDIO="$1"
STAKEHOLDER="$2"
MODEL="${3:-large-v3}"
```

```
DATE=$(date +%Y-%m-%d)
SLUG=$(echo "${STAKEHOLDER}" | tr '[:upper:] ' '[:lower:]-')
OUT_DIR="${HOME}/Documents/Vault/Transcripts"
NOTE_DIR="${HOME}/Documents/Vault/Interviews"
NOTE="${NOTE_DIR}/${DATE}-${SLUG}.md"
WORK_DIR=$(mktemp -d)
TEMPLATE="${HOME}/Documents/Vault/Templates/Interview.md"

mkdir -p "${OUT_DIR}" "${NOTE_DIR}"

# Activate venv
# shellcheck disable=SC1091
source "${HOME}/.venvs/whisperx/bin/activate"

whisperx "${AUDIO}" \
    --model "${MODEL}" \
    --diarize \
    --hf_token "${HF_TOKEN:?HF_TOKEN not set}" \
    --output_dir "${WORK_DIR}" \
    --output_format txt \
    --language en \
    --compute_type float16

BASE=$(basename "${AUDIO%.*}")
TXT="${WORK_DIR}/${BASE}.txt"

cp "${TXT}" "${OUT_DIR}/${DATE}-${SLUG}.txt"

sed -e "s/{{stakeholder}}/${STAKEHOLDER}/g" \
    -e "s/{{date}}/${DATE}/g" "${TEMPLATE}" > "${NOTE}"
echo "" >> "${NOTE}"
cat "${TXT}" >> "${NOTE}"

echo "${NOTE}"
EOF

chmod +x ~/.local/bin/transcribe-interview
```

The engine prints the path to the new note as its last line — the TUI reads that to show you where the file landed.

Quick smoke test:

```
transcribe-interview ~/transcripts/sample.m4a "Test Stakeholder"
```

Should print a path under `~/Documents/Vault/Interviews/`. Open that file in any text editor to confirm it has the template header at the top and the transcript at the bottom.

10. The gum TUI front-end

The TUI is `interview-transcribe`. It walks the user through file picking, stakeholder input, model choice, confirmation, and shows a spinner while the engine runs.

Save as `~/.local/bin/interview-transcribe` and `chmod +x` it.

```
cat > ~/.local/bin/interview-transcribe <<'EOF'
#!/usr/bin/env bash
# interview-transcribe — TUI front-end for the WhisperX + Obsidian pipeline.
set -euo pipefail

# --- prereq checks ---
for tool in gum transcribe-interview flatpak; do
    if ! command -v "$tool" &>/dev/null; then
        echo "missing: $tool" >&2; exit 1
    fi
done
if [[ -z "${HF_TOKEN:-}" ]]; then
```

```
    gum style --foreground 196 --bold \
        "HF_TOKEN is not set. Add it to ~/.bashrc and restart your shell."
    exit 1
fi

# --- banner ---
clear
gum style \
    --border double --margin "1 2" --padding "1 4" \
    --border-foreground 33 --foreground 33 --bold \
    "interview-transcribe" \
    "WhisperX + Obsidian, local + private"

# --- step 1: file ---
gum format -- "**Step 1 of 4.** Pick the audio file."
AUDIO=$(gum file "${HOME}/Downloads")
[[ -z "$AUDIO" ]] && { gum style --foreground 196 "No file chosen."; exit 1; }
gum style --foreground 244 "  -> $AUDIO"
echo

# --- step 2: stakeholder ---
gum format -- "**Step 2 of 4.** Who is this interview with?"
STAKEHOLDER=$(gum input --placeholder "e.g. Jane Doe" --width 50)
[[ -z "$STAKEHOLDER" ]] && STAKEHOLDER="unknown"
gum style --foreground 244 "  -> $STAKEHOLDER"
echo

# --- step 3: model ---
gum format -- "**Step 3 of 4.** Model size."
MODEL_LABEL=$(gum choose --header "Best quality first; lower if you hit OOM:" \
    "large-v3   (best quality, 8 GB+ VRAM)" \
    "large-v2   (good quality, 6 GB VRAM)" \
    "medium     (acceptable, 4 GB VRAM)" \
    "small      (fastest, 2 GB VRAM)")
MODEL=$(echo "$MODEL_LABEL" | awk '{print $1}')
gum style --foreground 244 "  -> $MODEL"
echo

# --- step 4: confirm ---
gum format -- "**Step 4 of 4.** Ready."
gum style --foreground 244 \
    "  File:        $(basename "$AUDIO")" \
    "  Stakeholder: $STAKEHOLDER" \
    "  Model:       $MODEL"
gum confirm "Transcribe now?" || { gum style --foreground 196 "Aborted."; exit 0; }
echo

# --- run ---
LOG=$(mktemp)
if gum spin --spinner dot \
    --title "Transcribing... (this can take a few minutes)" -- \
    bash -c "transcribe-interview '$AUDIO' '$STAKEHOLDER' '$MODEL' > '$LOG' 2>&1"; then
    NOTE=$(tail -n 1 "$LOG")
    gum style \
        --border rounded --margin "1 2" --padding "1 2" \
        --border-foreground 46 --foreground 46 \
        "Transcription complete." \
        "" \
        "Note: $NOTE"
    if gum confirm "Open in Obsidian now?"; then
        flatpak run md.obsidian.Obsidian "$NOTE" &>/dev/null &
    fi
else
    gum style --foreground 196 --bold "Transcription failed. Last lines of log:"
```

```
    tail -n 20 "$LOG"
    exit 1
fi
EOF

chmod +x ~/.local/bin/interview-transcribe
```

Launch it: type `interview-transcribe` in any terminal. The first prompt is a file browser starting in `~/Downloads` (use arrow keys, Enter to select, Esc to cancel).

> **Note.** The TUI calls the engine and parses the last line of stdout for the resulting note path. If you change the engine to print anything else as its final line, the TUI will misreport the note location. Keep the engine's last `echo` as the path.

11. Desktop launcher (.desktop file)

So the TUI shows up in the COSMIC app launcher next to your other apps. The launcher opens a terminal and runs `interview-transcribe` inside it.

```
mkdir -p ~/.local/share/applications
cat > ~/.local/share/applications/interview-transcribe.desktop <<'EOF'
[Desktop Entry]
Type=Application
Name=Interview Transcribe
Comment=Transcribe stakeholder interviews with speaker labels
Exec=cosmic-term -e bash -c 'interview-transcribe; echo; read -n 1 -s -r -p "Press any key to close."'
Icon=audio-input-microphone
Terminal=false
Categories=AudioVideo;Utility;Office;
Keywords=transcribe;whisper;interview;notes;
EOF

# Refresh the app database
update-desktop-database ~/.local/share/applications 2>/dev/null || true
```

Press Super, type "Interview" — Interview Transcribe should appear. Click it: a cosmic-term window opens with the TUI running. After the TUI exits, the terminal pauses on "Press any key to close" so you can read any error output before it disappears.

> **Note.** If you use a different terminal (alacritty, gnome-terminal, ghostty), replace `cosmic-term` in the Exec line. The pattern is: `-e bash -c '...'`. Confirm your terminal's flag for "run this command" — most use `-e` but some use `--command`.

12. Optional — local summarization with Ollama

If you want a one-paragraph summary and action-item extraction inline with the transcript, Ollama runs an LLM locally — no audio or text leaves your machine.

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Pull a model. llama3.1:8b is a good balance for an 8 GB GPU.
ollama pull llama3.1:8b

ollama run llama3.1:8b "Say hello in five words."
```

To wire summarization into the engine, edit `~/.local/bin/transcribe-interview` and insert this block right before the final `echo "${NOTE}"`:

```
# --- optional Ollama summary ---
PROMPT="You are summarizing a stakeholder interview transcript. "
PROMPT+="Produce: (1) a three-sentence summary, (2) up to 5 key claims with "
PROMPT+="the speaker label, (3) any explicit action items. Be concise. Transcript:"

SUMMARY_FILE=$(mktemp)
ollama run llama3.1:8b "${PROMPT}\n\n$(cat "${TXT}")" > "${SUMMARY_FILE}" 2>/dev/null

# Inject under the Summary heading
```

```
python3 - <<'PY' "${NOTE}" "${SUMMARY_FILE}"
import sys, pathlib
note = pathlib.Path(sys.argv[1])
summary = pathlib.Path(sys.argv[2]).read_text().strip()
text = note.read_text()
text = text.replace("## Summary\n\n\n", f"## Summary\n\n{summary}\n\n", 1)
note.write_text(text)
PY
```

Run the TUI again — the resulting note will have a populated Summary section.

Part 3 — Use the app

13. Daily workflow

Once set up, an interview run looks like this:

- **Record.** Phone voice memo, an external recorder (Plaud, Zoom H-series), or a Zoom/Meet call recorded to disk. Save anywhere — `~/Downloads` is fine.
- **Launch.** Open a terminal and type `interview-transcribe`, or press Super and click Interview Transcribe.
- **Walk the prompts.** Pick file → stakeholder name → model size → confirm. About 20 seconds of input from you.
- **Wait.** On a recent NVIDIA card, a 45-minute interview takes 2–4 minutes. The spinner tells you it's working.
- **Synthesize.** When the TUI asks "Open in Obsidian now?", say yes. The new note opens with the raw transcript at the bottom and an empty synthesis section at the top. Read, fill in **Key claims** and **Open questions**, tag and link to your project page.
- **Retire the audio.** Delete the source file unless you have a specific reason to keep it. The transcript is the record.

14. Consent and storage

Recording stakeholders without their knowledge is a legal problem in many US states and almost all of Europe. Two habits cover the common cases:

- Add a line to your meeting invite: *"I'll record this session for my own notes; the recording stays on my laptop and is deleted after I've written it up."*
- Ask once at the top of the call: *"Mind if I record this so I can focus on listening?"* Pause for an answer. If anyone hesitates, take notes by hand instead.

The local-only nature of this pipeline is the right answer when stakeholders ask "where does this audio go?" Nothing leaves your machine — no cloud transcription vendor, no OpenAI, no Anthropic. Worth saying out loud at the start of the call.

15. Troubleshooting

"CUDA out of memory"

Your GPU can't fit the model. In the TUI, choose a smaller model on Step 3. If you're calling the engine directly, lower `--batch_size` (default 16, try 8, 4, 2) or drop `--model` from `large-v3` to `large-v2`, then `medium`. As a last resort, transcribe on CPU with `--device cpu --compute_type int8` — slow but works.

"Could not download pyannote/..."

You didn't accept the gated-model terms. Re-do step 5 in full. Both pages must show "You have access to this model" with your account logged in.

Diarization labels are wrong / merged speakers

Pyannote does best with clean audio. Quick wins: place the mic equidistant from speakers, ask people to introduce themselves at the start ("This is Jane — I lead the editorial team"), and consider editing the engine to pass `--min_speakers N --max_speakers N` when you know the count in advance.

ffmpeg complains about codec

Some phone recordings use proprietary codecs Pop!_OS doesn't ship. Re-encode to WAV: `ffmpeg -i input.m4a -ac 1 -ar 16000 output.wav`, then transcribe the WAV.

Obsidian won't open the vault folder

Flatpak sandboxes filesystem access. The fast fix is a one-line override:

```
flatpak override --user --filesystem=home md.obsidian.Obsidian
```

Or install **Flatseal** for a GUI: `flatpak install -y flathub com.github.tchx84.Flatseal`.

The app grid icon doesn't appear

Run `update-desktop-database ~/.local/share/applications` again, then log out and back in. If it still doesn't show, check that the `.desktop` file has no syntax errors with `desktop-file-validate ~/.local/share/applications/interview-transcribe.desktop`.

TUI hangs after "Transcribing..."

The engine is running but slowly — verify with `nvidia-smi` in another terminal that the GPU is busy. If it's idle, the model fell back to CPU; check that the venv has the CUDA build of PyTorch (step 3 verification).

Token leaks into git commits

If you keep notes in a git repo, add `.env` and any shell-rc files to `.gitignore`. The `HF_TOKEN` lives in `~/.bashrc` — outside the vault — but worth saying out loud.

Appendix A — Versions this guide targets

Component	Target version
Pop!_OS	24.04 LTS (COSMIC Epoch 1, released Dec 2025)
Kernel	6.17.x
NVIDIA driver	560.x or newer
Python	3.12 (system default on 24.04)
PyTorch	2.4+ with CUDA 12.4 wheel (cu124)
WhisperX	3.1+
pyannote.audio	3.3+ (segmentation-3.0, diarization-3.1)
Whisper model	large-v3 default; large-v2/medium for lower VRAM
gum	Latest from repo.charm.sh/apt
Obsidian	Flatpak from Flathub, verified by Obsidian team
Ollama	Latest stable; llama3.1:8b suggested default
ffmpeg	6.x or newer (Pop!_OS default)

Appendix B — Disk and download footprint

Plan for roughly:

- **~5 GB** for the Python venv + PyTorch + WhisperX + pyannote
- **~3 GB** for the Whisper large-v3 model (one-time download)
- **~1 GB** for pyannote diarization models

- **~5 GB** for an 8B Ollama model, if you add Ollama
- **~250 MB** for the Obsidian Flatpak (plus shared runtimes already on disk)

All under `~/.cache`, `~/.venvs`, `~/.ollama`, and the Flatpak directories. Easy to clean up later if you change your mind.

Appendix C — When something here goes stale

The pieces that move fastest, in rough order:

- **PyTorch CUDA wheel index** — the cu124 URL might roll forward to cu126/cu128. Check pytorch.org/get-started/locally for the current line.
- **pyannote model versions** — currently 3.x; if a 4.x ships, accept its terms and WhisperX picks it up after `pip install --upgrade whisperx`.
- **Ollama models** — llama3.1 will eventually be superseded; `ollama pull` the newer one.
- **gum** — Charm rev the syntax occasionally. If a `gum` command in the TUI errors after an update, check `gum --help` for renamed flags.
- **Pop!_OS itself** — the next LTS lands April 2026 (26.04) and changes very little for this stack.

If a step here breaks, check the project README — WhisperX on GitHub, Obsidian's obsidian.md/help, Charm's docs, and System76 support are the load-bearing sources.